

4장 서버 프로그램 구현



340165

20.5



089 모듈화

(A)

- 모듈화(Modularity)는 소프트웨어의 성능 향상, 시스템의 수정 및 재사용, 유지 관리 등이 용이하도록 시스템의 기능들을 모듈 단위로 나누는 것을 의미한다.
- 모듈화는 모듈 간 결합도(Coupling)의 최소화와 모듈 내 요소들의 응집도(Cohesion)를 최대화하는 것이 목표이다.

340166

필기 24.7, 21.8



090 추상화

(C)

- 추상화(Abstraction)는 문제의 전체적이고 포괄적인 개념을 설계한 후 차례로 세분화하여 구체화시키 나가는 것이다.
- 추상화의 유형

과정 추상화	자세한 수행 과정을 정의하지 않고, 전반적인 흐름만 파악할 수 있게 설계하는 방법
자료 추상화	데이터의 세부적인 속성이나 용도를 정의하지 않고, 데이터 구조를 대표할 수 있는 표현으로 대체하는 방법
제어 추상화	이벤트 발생의 정확한 절차나 방법을 정의하지 않고, 대표할 수 있는 표현으로 대체하는 방법

340168

필기 25.5, 24.5, 21.8, 21.5



091 정보 은닉

(C)

- 정보 은닉(Information Hiding)은 한 모듈 내부에 포함된 절차와 자료들의 정보가 감추어져 다른 모듈이 접근하거나 변경하지 못하도록 하는 기법이다.
- 정보 은닉을 통해 모듈을 독립적으로 수행할 수 있다.
- 하나의 모듈이 변경되더라도 다른 모듈에 영향을 주지 않으므로 수정, 시험, 유지보수가 용이하다.

340171

필기 23.2, 20.8



092 협약(Contract)에 의한 설계

(C)

- 협약에 의한 설계는 컴포넌트를 설계할 때 클래스에 대한 여러 가정을 공유할 수 있도록 명세한 것이다.
- 컴포넌트에 대한 정확한 인터페이스를 명세한다.
- 명세에 포함될 조건

선행 조건 (Precondition)	오퍼레이션이 호출되기 전에 참이 되어야 할 조건
결과 조건 (Postcondition)	오퍼레이션이 수행된 후 만족되어야 할 조건
불변 조건 (Invariant)	오퍼레이션이 실행되는 동안 항상 만족되어야 할 조건

340175

필기 25.8, 24.7, 23.7, 23.2, 22.7, 21.8, 21.5, 20.9



093 파이프-필터 패턴

(B)

- 파이프-필터 패턴(Pipe-Filter Pattern)은 데이터 스트림 절차의 각 단계를 필터로 캡슐화하여 파이프를 통해 전송하는 패턴이다.
- 앞 시스템의 처리 결과물을 파이프를 통해 전달받아 처리한 후 그 결과물을 다시 파이프를 통해 다음 시스템으로 넘겨주는 패턴을 반복한다.
- 데이터 변환, 버퍼링, 동기화 등에 주로 사용된다.
- 대표적으로 UNIX의 셸(Shell)이 있다.

340177

필기 24.5, 23.5, 23.2, 21.8



094 기타 패턴

(B)

필기 24.5, 23.5, 23.2, 21.8

마스터-슬레이브 패턴(Master-Slave Pattern)

슬레이브 컴포넌트에서 처리된 결과물을 다시 돌려받는 방식으로 작업을 수행하는 패턴이다.

예 장애 허용 시스템, 병렬 컴퓨팅 시스템

브로커 패턴(Broker Pattern)

사용자가 원하는 서비스와 특성을 브로커 컴포넌트에 요청하면 브로커 컴포넌트가 요청에 맞는 컴포넌트와 사용자를 연결해주는 패턴이다.

예 분산 환경 시스템

피어-투-피어 패턴(Peer-To-Peer Pattern)

피어(Peer)라 불리는 하나의 컴포넌트가 클라이언트가 될수도, 서버가 될 수도 있는 패턴이다.

예 파일 공유 네트워크

이벤트-버스 패턴(Event-Bus Pattern)

소스가 특정 채널에 이벤트 메시지를 발행(Publish)하면, 해당 채널을 구독(Subscribe)한 리스너(Listener)들이 메시지를 받아 이벤트를 처리하는 패턴이다.

예 알림 서비스

블랙보드 패턴(Blackboard Pattern)

모든 컴포넌트들이 공유 데이터 저장소와 블랙보드 컴포넌트에 접근이 가능한 패턴이다.

예 음성 인식, 차량 식별, 신호 해석

필기 21.8

인터프리터 패턴(Interpreter Pattern)

프로그램 코드의 각 라인을 수행하는 방법을 지정하고, 기호마다 클래스를 갖도록 구성된 패턴이다.

예 번역기, 컴파일러, 인터프리터

340180



필기 25.8, 25.5, 25.2, 24.2, 23.5, 22.3, 21.8, 21.5, 20.8, 20.6

095 클래스



- 클래스(Class)는 공통된 속성과 연산을 갖는 객체의 집합이다.
- 각각의 객체들이 갖는 속성과 연산을 정의하고 있는 틀이다.
- 클래스에 속한 각각의 객체를 인스턴스(Instance)라고 한다.

440149



필기 25.2, 23.2, 22.7, 21.5

096 메시지



- 메시지(Message)는 객체들 간의 상호작용에 사용되는 수단으로, 객체의 동작이나 연산을 일으키는 외부의 요구 사항이다.
- 메시지를 받은 객체는 대응하는 연산을 수행하여 예상된 결과를 반환한다.

340182



필기 24.5, 24.2, 23.5, 21.8, 21.3, 20.9, 20.8

097 캡슐화



- 캡슐화(Encapsulation)는 외부에서의 접근을 제한하기 위해 인터페이스를 제외한 세부 내용을 은닉하는 것이다.
- 캡슐화된 객체는 외부 모듈의 변경으로 인한 파급 효과가 적다.
- 객체들 간에 메시지를 주고받을 때 상대 객체의 세부 내용은 알 필요가 없으므로 인터페이스가 단순해지고, 객체 간의 결합도가 낮아진다.

340186



필기 23.7, 21.8, 21.3

098 객체지향 분석



- 객체지향 분석(OOA; Object Oriented Analysis)은 사용자의 요구사항과 관련된 객체, 속성, 연산, 관계 등을 정의하여 모델링하는 작업이다.
- 개발을 위한 업무를 객체와 속성, 클래스와 멤버, 전체와 부분 등으로 나누어서 분석한다.
- 클래스를 식별하는 것이 객체지향 분석의 주요 목적이다.



099 객체지향 분석의 방법론

(C)

- Rumbaugh(럼바우) 방법 : 분석 활동을 객체 모델, 동적 모델, 기능 모델로 나누어 수행함
- Booch(부치) 방법 : 미시적(Micro) 개발 프로세스와 거시적(Macro) 개발 프로세스를 모두 사용하며, 클래스와 객체들을 분석 및 식별하고 클래스의 속성과 연산을 정의함
- Jacobson 방법 : 유스케이스(Use Case)를 강조하여 사용함
- Coad와 Yourdon 방법 : E-R 다이어그램을 사용하여 객체의 행위를 모델링하며, 객체 식별, 구조 식별, 주제 정의, 속성과 인스턴스 연결 정의, 연산과 메시지 연결 정의 등의 과정으로 구성함
- Wirfs-Brock 방법 : 분석과 설계 간의 구분이 없고, 고객 명세서를 평가해서 설계 작업까지 연속적으로 수행함



100 럼바우의 분석 기법

(A)

- 럼바우(Rumbaugh)의 분석 기법은 모든 소프트웨어 구성 요소를 그래픽 표기법을 이용하여 모델링하는 기법이다.
- 객체 모델링 기법(OMT, Object-Modeling Technique)이라고도 한다.
- 분석 활동은 '객체 모델링 → 동적 모델링 → 기능 모델링' 순으로 이루어 진다.
 - 객체 모델링(Object Modeling) : 정보 모델링(Information Modeling)이라고도 하며, 시스템에서 요구되는 객체를 찾아내어 속성과 연산 식별 및 객체들 간의 관계를 규정하여 객체 다이어그램으로 표시하는 모델링
 - 동적 모델링(Dynamic Modeling) : 상태 다이어그램을 이용하여 시간의 흐름에 따른 객체들 간의 제어 흐름, 상호 작용, 동작 순서 등의 동적인 행위를 표현하는 모델링
 - 기능 모델링(Functional Modeling) : 자료 흐름도(DFD)를 이용하여 다수의 프로세스들 간의 자료 흐름을 중심으로 처리 과정을 표현한 모델링



101 객체지향 설계 원칙(SOLID 원칙)

(A)

- 단일 책임 원칙(SRP) : 객체는 단 하나의 책임만 가져야 한다는 원칙
- 개방-폐쇄 원칙(OCP) : 기존의 코드를 변경하지 않고 기능을 추가할 수 있도록 설계해야 한다는 원칙
- 리스코프 치환 원칙(LSP) : 자식 클래스는 최소한 부모 클래스의 기능은 수행할 수 있어야 한다는 원칙
- 인터페이스 분리 원칙(ISP) : 자신이 사용하지 않는 인터페이스와 의존 관계를 맺거나 영향을 받지 않아야 한다는 원칙
- 의존 역전 원칙(DIP) : 의존 관계 성립 시 추상성이 높은 클래스와 의존 관계를 맺어야 한다는 원칙



102 모듈

(B)

- 모듈(Module)은 모듈화를 통해 분리된 시스템의 각 기능으로, 서브루틴, 서브시스템, 소프트웨어 내의 프로그램, 작업 단위 등을 의미한다.
- 모듈의 기능적 독립성은 소프트웨어를 구성하는 각 모듈의 기능이 서로 독립됨을 의미한다.
- 하나 또는 몇 개의 논리적인 기능을 수행하기 위한 명령어들의 집합이라고도 할 수 있다.
- 모듈의 독립성은 결합도(Coupling)와 응집도(Cohesion)에 의해 측정된다.

340191

필기 25.2, 24.5, 23.2, 21.8, 21.5, 21.3, 20.8, 20.6



103 결합도

(B)

- 결합도(Coupling)는 모듈 간에 상호 의존하는 정도 또는 두 모듈 사이의 연관 관계이다.
- 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮다.
- 결합도의 종류와 강도

내용 결합도	공통 결합도	외부 결합도	제어 결합도	스탬프 결합도	자료 결합도
--------	--------	--------	--------	---------	--------

결합도 강함 ← → 결합도 약함

340192

25.4, 24.7, 21.10, 21.4, 필기 25.8, 23.7, 23.2, 22.7, 22.4, 20.9, 20.8



104 결합도의 종류

(A)

25.4, 21.4, 필기 25.8, 23.7, 23.2, 22.7, 22.4, 20.9

내용 결합도(Content Coupling)

한 모듈이 다른 모듈의 내부 기능 및 그 내부 자료를 직접 참조하거나 수정할 때의 결합도이다.

25.4, 21.4, 필기 23.7, 23.2, 20.9

공통(공유) 결합도(Common Coupling)

- 공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도이다.
- 파라미터가 아닌 모듈 밖에 선언된 전역 변수를 사용하여 전역 변수를 갱신하는 방식으로 상호작용하는 때의 결합도이다.

필기 25.8, 22.7

외부 결합도(External Coupling)

어떤 모듈에서 선언한 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도이다.

24.7, 21.10, 필기 20.8

제어 결합도(Control Coupling)

- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도이다.
- 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 된다.

25.4, 21.4, 필기 25.8, 23.2, 22.7, 22.4, 20.9

스탬프(검인) 결합도(Stamp Coupling)

모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도이다.

필기 23.7, 23.2, 22.7, 20.9

자료 결합도(Data Coupling)

모듈 간의 인터페이스가 자료 요소로만 구성될 때의 결합도이다.

340193

24.4, 필기 25.8, 24.5, 23.5, 23.2, 22.4, 21.5, 21.3, 20.6



105 응집도

(A)

- 응집도(Cohesion)는 모듈의 내부 요소들이 서로 관련되어 있는 정도이다.
- 응집도가 강할수록 품질이 높고, 약할수록 품질이 낮다.
- 응집도의 종류와 강도

기능적 응집도	순차적 응집도	교환적 응집도	절차적 응집도	시간적 응집도	논리적 응집도	우연적 응집도
---------	---------	---------	---------	---------	---------	---------

응집도 강함 ← → 응집도 약함

340194

24.7, 21.7, 필기 21.8, 20.9, 20.8



106 응집도의 종류

(A)

21.7

기능적 응집도(Functional Cohesion)

모듈 내부의 모든 기능 요소들이 단일 문제와 연관되어 수행될 경우의 응집도이다.

24.7

순차적 응집도(Sequential Cohesion)

모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도이다.

21.7

교환(통신)적 응집도(Communication Cohesion)

동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성 요소들이 모였을 경우의 응집도이다.

절차적 응집도(Procedural Cohesion)

모듈이 다수의 관련 기능을 가질 때 모듈 안의 구성 요소들이 그 기능을 순차적으로 수행할 경우의 응집도이다.

시간적 응집도(Temporal Cohesion)

특정 시간에 처리되는 몇 개의 기능을 모아 하나의 모듈로 작성할 경우의 응집도이다.

논리적 응집도(Logical Cohesion)

유사한 성격을 갖거나 특정 형태로 분류되는 처리 요소들로 하나의 모듈이 형성되는 경우의 응집도이다.

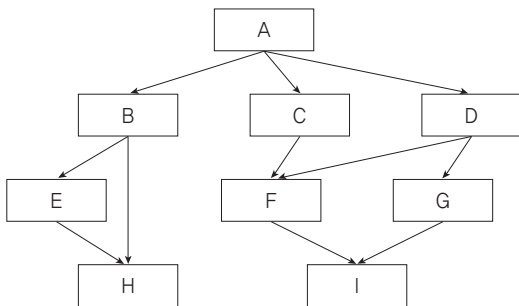
우연적 응집도(Coincidental Cohesion)

모듈 내부의 각 구성 요소들이 서로 관련 없는 요소로만 구성된 경우의 응집도이다.

**107 팬인 / 팬아웃**

- 팬인(Fan-In)은 어떤 모듈을 제어하는 모듈의 수이다.
- 팬아웃(Fan-Out)은 어떤 모듈에 의해 제어되는 모듈의 수를 의미한다.

예제 다음의 시스템 구조도에서 각 모듈의 팬인(Fan-In)과 팬아웃(Fan-Out)을 구하시오.

**해설**

- 팬인(Fan-In) : A는 0, B · C · D · E · G는 1, F · H · I는 2
- 팬아웃(Fan-Out) : H · I는 0, C · E · F · G는 1, B · D는 2, A는 3

**108 N-S 차트**

- N-S 차트(Nassi-Schneiderman Chart)는 논리의 기술에 중점을 두고 도형을 이용해 표현하는 방법이다.
- GOTO나 화살표를 사용하지 않는다.
- 연속, 선택 및 다중 선택, 반복의 3가지 제어 논리 구조로 표현한다.
- 조건이 복합되어 있는 곳의 처리를 시각적으로 명확히 식별하는 데 적합하다.

**109 IPC**

- IPC(Inter-Process Communication)는 모듈 간 통신 방식을 구현하기 위해 사용되는 대표적인 프로그래밍 인터페이스 집합이다.
- 복수의 프로세스를 수행하며 이뤄지는 프로세스 간 통신까지 구현이 가능하다.
- IPC의 대표 메소드 5가지
 - 공유 메모리(Shared Memory)
 - 소켓(Socket)
 - 세마포어(Semaphores)
 - 파이프와 네임드 파이프(Pipes & named Pipes)
 - 메시지 큐잉(Message Queueing)



110 테스트 케이스



- 테스트 케이스(Test Case)는 구현된 소프트웨어가 사용자의 요구사항을 정확하게 준수했는지를 확인하기 위한 테스트 항목에 대한 명세서이다.
- ISO/IEC/IEEE 29119-3 표준에 따른 테스트 케이스의 구성 요소
 - 식별자 : 항목 식별자, 일련번호
 - 테스트 항목 : 테스트 대상(모듈 또는 기능)
 - 입력 명세 : 테스트 데이터 또는 테스트 조건
 - 출력 명세 : 테스트 케이스 수행 시 예상되는 출력 결과
 - 환경 설정 : 필요한 하드웨어나 소프트웨어의 환경
 - 특수 절차 요구 : 테스트 케이스 수행 시 특별히 요구되는 절차
 - 의존성 기술 : 테스트 케이스 간의 의존성



111 재사용



- 재사용(Reuse)은 이미 개발된 기능들을 새로운 시스템이나 기능 개발에 사용하기 적합하도록 최적화하는 작업이다.
- 새로 개발하는데 필요한 비용과 시간을 절약할 수 있다.
- 누구나 이해할 수 있고 사용이 가능하도록 사용법을 공개해야 한다.
- 재사용 규모에 따른 분류

필기 20.9 함수와 객체	클래스나 메소드 단위의 소스 코드를 재사용함
필기 20.9 컴포넌트	컴포넌트 자체에 대한 수정 없이 인터페이스를 통해 통신하는 방식으로 재사용함
필기 20.9 애플리케이션	공통된 기능들을 제공하는 애플리케이션을 공유하는 방식으로 재사용함



112 코드의 종류



필기 23.7, 23.2, 20.6

순차 코드(Sequence Code)

자료의 발생 순서, 크기 순서 등 일정 기준에 따라서 최초의 자료부터 차례로 일련번호를 부여하는 방법으로, 순서 코드 또는 일련번호 코드라고도 한다.

예 1, 2, 3, 4, ...

블록 코드(Block Code)

코드화 대상 항목 중에서 공통성이 있는 것끼리 블록으로 구분하고, 각 블록 내에서 일련번호를 부여하는 방법으로, 구분 코드라고도 한다.

예 1001~1100 : 총무부, 1101~1200 : 영업부

10진 코드(Decimal Code)

코드화 대상 항목을 0~9까지 10진 분할하고, 다시 각각에 대하여 10진 분할하는 방법을 필요한 만큼 반복하는 방법으로, 도서 분류식 코드라고도 한다.

예 1000 : 공학, 1100 : 소프트웨어 공학, 1110 : 소프트웨어 설계

그룹 분류 코드(Group Classification Code)

코드화 대상 항목을 일정 기준에 따라 대분류, 중분류, 소분류 등으로 구분하고, 각 그룹 안에서 일련번호를 부여하는 방법이다.

예 1-01-001 : 본사-총무부-인사계, 2-01-001 : 지사-총무부-인사계

연상 코드(Mnemonic Code)

코드화 대상 항목의 명칭이나 약호와 관계있는 숫자나 문자, 기호를 이용하여 코드를 부여하는 방법이다.

예 TV-40 : 40인치 TV, L-15-220 : 15W 220V의 램프

필기 23.2, 20.9

표의 숫자 코드(Significant Digit Code)

코드화 대상 항목의 성질, 즉 길이, 넓이, 부피, 지름, 높이 등의 물리적 수치를 그대로 코드에 적용시키는 방법으로, 유효 숫자 코드라고도 한다.

예 120-720-1500 : 두께×폭×길이가 120×720×1500인 강판

합성 코드(Combined Code)

필요한 기능을 하나의 코드로 수행하기 어려운 경우 2개 이상의 코드를 조합하여 만드는 방법이다.

예 연상 코드 + 순차 코드

KE-711 : 대한항공 711기, AC-253 : 에어캐나다 253기

340204



20.11, 필기 25.8, 24.7, 23.5, 22.3, 21.8, 20.9, 20.8

113 디자인 패턴



- 디자인 패턴(Design Pattern)은 모듈 간의 관계 및 인터페이스를 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미한다.
- 문제 및 배경, 실제 적용된 사례, 재사용이 가능한 샘플 코드 등으로 구성되어 있다.
- GOF의 디자인 패턴은 생성 패턴, 구조 패턴, 행위 패턴으로 구분된다.

340205



24.4, 21.10, 필기 25.8, 25.2, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, 22.3, 21.8, ...

114 생성 패턴



생성 패턴(Creational Pattern)은 클래스나 객체의 생성과 참조 과정을 정의하는 패턴이다.

24.4, 필기 24.5, 22.3, 21.3

추상 팩토리(Abstract Factory)

- 구체적인 클래스에 의존하지 않고, 인터페이스를 통해 서로 연관·의존하는 객체들의 그룹으로 생성하여 추상적으로 표현하는 패턴이다.
- 연관된 서브 클래스를 묶어 한 번에 교체하는 것이 가능하다.

필기 21.3

빌더(Builder)

- 작게 분리된 인스턴스를 건축 하듯이 조합하여 객체를 생성하는 패턴이다.

- 객체의 생성 과정과 표현 방법을 분리하고 있어, 동일한 객체 생성에서도 서로 다른 결과를 만들어 낼 수 있다.

21.10, 필기 25.5, 23.7, 23.2, 21.5, 20.8

팩토리 메소드(Factory Method)

- 객체 생성을 서브 클래스에서 처리하도록 분리하여 캡슐화한 패턴이다.
- 상위 클래스에서 인터페이스만 정의하고 실제 생성은 서브 클래스가 담당한다.
- 가상 생성자(Virtual Constructor) 패턴이라고도 한다.

필기 23.2, 21.5, 20.8

프로토타입(Prototype)

- 원본 객체를 복제하는 방법으로 객체를 생성하는 패턴이다.
- 일반적인 방법으로 객체를 생성하며, 비용이 큰 경우 주로 이용한다.

필기 25.2, 24.2, 23.5, 22.7, 21.8, 21.5, 21.3

싱글톤(Singleton)

- 하나의 객체를 생성하면 생성된 객체를 어디서든 참조할 수 있지만, 여러 프로세스가 동시에 참조할 수는 없는 패턴이다.
- 클래스 내에서 인스턴스가 하나뿐임을 보장하며, 불필요한 메모리 낭비를 최소화 할 수 있다.

440174



25.7, 25.4, 23.4, 22.10, 필기 25.8, 25.5, 23.2, 22.4, 21.5

115 구조 패턴



구조 패턴(Structural Pattern)은 구조가 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하여 더 큰 구조로 만드는 패턴이다.

25.4, 필기 25.8, 25.5, 23.2, 22.4, 21.5

어댑터(Adapter)

- 호환성이 없는 클래스들의 인터페이스를 다른 클래스가 이용할 수 있도록 변환해주는 패턴이다.
- 기존의 클래스를 이용하고 싶지만 인터페이스가 일치하지 않을 때 이용한다.

브리지(Bridge)

- 구현부에서 추상층을 분리하여 서로가 독립적으로 확장할 수 있도록 구성한 패턴이다.
- 기능과 구현을 두 개의 별도 클래스로 구현한다.

필기 25.8, 23.2

컴포지트(Composite)

- 여러 객체를 가진 복합 객체와 단일 객체를 구분 없이 다루고자 할 때 사용하는 패턴이다.
- 객체들을 트리 구조로 구성하여 디렉터리 안에 디렉터리가 있듯이 복합 객체 안에 복합 객체가 포함되는 구조를 구현할 수 있다.

필기 25.5, 23.2

데코레이터(Decorator)

- 객체 간의 결합을 통해 능동적으로 기능들을 확장할 수 있는 패턴이다.
- 임의의 객체에 부가적인 기능을 추가하기 위해 다른 객체들을 덧붙이는 방식으로 구현한다.

퍼싸드(Facade)

- 복잡한 서브 클래스들을 피해 더 상위 인터페이스를 구성함으로써 서브 클래스들의 기능을 간편하게 사용할 수 있도록 하는 패턴이다.
- 서브 클래스들 사이의 통합 인터페이스를 제공하는 Wrapper 객체가 필요하다.

플라이웨이트(Flyweight)

- 인스턴스가 필요할 때마다 매번 생성하는 것이 아니고 가능한 한 공유해서 사용함으로써 메모리를 절약하는 패턴이다.
- 다수의 유사 객체를 생성하거나 조작할 때 유용하게 사용할 수 있다.

25.7, 23.4, 필기 25.5, 22.4

프록시(Proxy)

- 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하는 패턴으로, 대리자라고도 불린다.
- 내부에서는 객체 간의 복잡한 관계를 단순하게 정리해 주고, 외부에서는 객체의 세부적인 내용을 숨겨주는 역할을 수행한다.

340207



24.10, 24.7, 22.10, 21.7, 20.7, 필기 25.5, 24.2, 23.5, 23.2, 21.8, 21.5, 20.8, 20.6

116 행위 패턴

행위 패턴(Behavioral Pattern)은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴이다.

책임 연쇄(Chain of Responsibility)

- 요청을 처리할 수 있는 객체가 둘 이상 존재하여 한 객체가 처리하지 못하면 다음 객체로 넘어가는 형태의 패턴이다.
- 요청을 처리할 수 있는 각 객체들이 고리(Chain)로 묶여 있어 요청이 해결될 때까지 고리를 따라 책임이 넘어간다.

필기 20.8

커맨드(Command)

- 요청을 객체의 형태로 캡슐화하여 재이용하거나 취소할 수 있도록 요청에 필요한 정보를 저장하거나 로그에 남기는 패턴이다.
- 요청에 사용되는 각종 명령어들을 추상 클래스와 구체 클래스로 분리하여 단순화한다.

인터프리터(Interpreter)

- 언어에 문법 표현을 정의하는 패턴이다.
- SQL이나 통신 프로토콜과 같은 것을 개발할 때 사용한다.

24.7

반복자(Iterator)

- 자료 구조와 같이 접근이 잦은 객체에 대해 동일한 인터페이스를 사용하도록 하는 패턴이다.
- 내부 표현 방법의 노출 없이 순차적인 접근이 가능하다.

필기 23.2, 21.5

중재자(Mediator)

- 수많은 객체들 간의 복잡한 상호작용(Interface)을 캡슐화하여 객체로 정의하는 패턴이다.
- 객체 사이의 의존성을 줄여 결합도를 감소시킬 수 있다.

메멘토(Memento)

- 특정 시점에서의 객체 내부 상태를 객체화함으로써 이후 요청에 따라 객체를 해당 시점의 상태로 돌릴 수 있는 기능을 제공하는 패턴이다.
- **Ctrl+Z**와 같은 되돌리기 기능을 개발할 때 주로 이용한다.

필기 22.10, 20.7, 필기 20.8

옵서버(Observer)

- 한 객체의 상태가 변화하면 객체에 상속되어 있는 다른 객체들에게 변화된 상태를 전달하는 패턴이다.
- 일대다의 의존성을 정의한다.
- 주로 분산된 시스템 간에 이벤트를 생성·발행(Publish)하고, 이를 수신(Subscribe)해야 할 때 이용한다.

필기 20.8

상태(State)

- 객체의 상태에 따라 동일한 동작을 다르게 처리해야 할 때 사용하는 패턴이다.
- 객체 상태를 캡슐화하고 이를 참조하는 방식으로 처리한다.

필기 24.2, 21.8

전략(Strategy)

- 동일한 계열의 알고리즘들을 개별적으로 캡슐화하여 상호 교환할 수 있게 정의하는 패턴이다.
- 클라이언트는 독립적으로 원하는 알고리즘을 선택하여 사용할 수 있으며, 클라이언트에 영향 없이 알고리즘의 변경이 가능하다.

필기 23.2

템플릿 메소드(Template Method)

- 상위 클래스에서 골격을 정의하고, 하위 클래스에서 세부 처리를 구체화하는 구조의 패턴이다.
- 유사한 서브 클래스를 묶어 공통된 내용을 상위 클래스에서 정의함으로써 코드의 양을 줄이고 유지보수를 용이하게 해준다.

필기 25.5, 20.6

방문자(Visitor)

- 각 클래스들의 데이터 구조에서 처리 기능을 분리하여 별도의 클래스로 구성하는 패턴이다.
- 분리된 처리 기능은 각 클래스를 방문(Visit)하여 수행한다.